

UAP Whitepaper

Abstract

This document proposes a reference architecture for an enterprise-grade agentic AI platform that supports multi-step workflow automation in regulated environments. The design separates a shared Tool Plane (standardized, governed enterprise capabilities) from a federated Agent Plane (domain and specialist agents deployed across clouds). The paper details memory and state management (short-term, episodic, and long-term), and walks through a Pre-Service Prior Authorization use case end-to-end.

Vision: Enable safe, scalable agentic automation across the enterprise by treating tools as shared platform products and agents as federated services. The platform should make it easy to build domain agents (pre-service, claims, operations) that are evidence-grounded, policy-compliant, observable, and reusable.

- Treats **agents as first-class enterprise citizens**
- Enables **secure, governed, multi-agent collaboration**
- Abstracts LLMs behind a controlled, extensible interface
- Integrates deeply with enterprise APIs, data, and events
- Supports **human-in-the-loop** and continuous learning

Three core challenges:

1. **Reusability:** Agents and enterprise capabilities must be reusable across domains, not re-built per use case.
2. **Governance and compliance:** Every action must be auditable, policy-bound, and privacy-aware.
3. **Operationalization:** Systems must be observable, cost-controlled, resilient, and safe at production scale.

Design Principles

1. **Separation of concerns** -Agents handle planning/reasoning; tools handle enterprise actions; governance is enforced consistently.

2. **Reusability by default** -Any agent can call any approved tool; any orchestrator can reuse specialized agents.
3. **Evidence-first** -Every outcome is grounded in policy/knowledge and tool outputs; citations are traceable.
4. **Least privilege + data minimization** -Only required attributes flow across boundaries; avoid persisting PHI in caches.
5. **Federated runtime, centralized governance** -Allow multi-cloud agent deployment while centralizing policies, catalogs, and audit patterns.
6. **Operational readiness** -Observability, SLOs, cost budgets, retries, and fallbacks are mandatory.

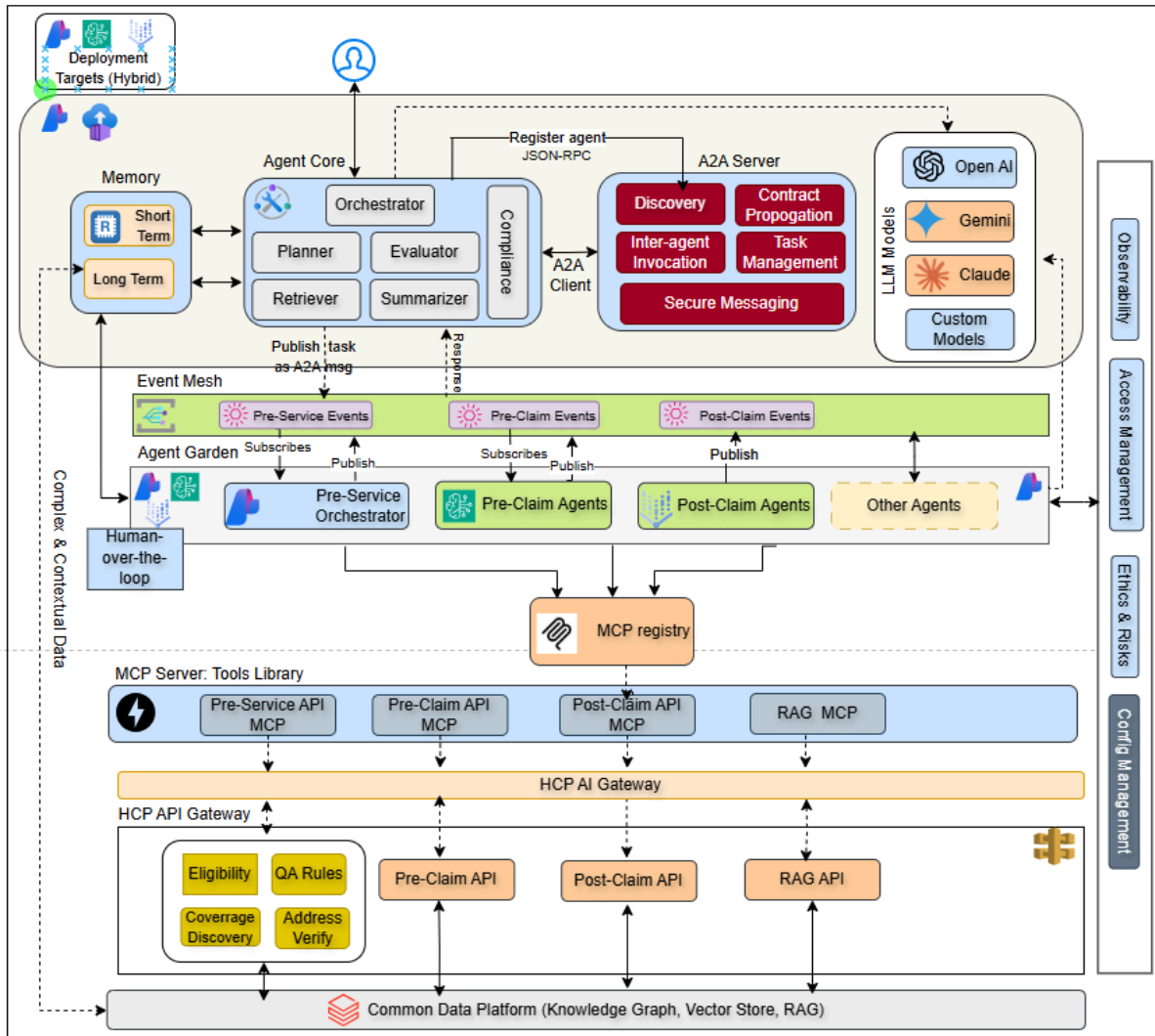
AI Platform architecture is based on three fundamentals:

- **Tool Plane (Shared):** Enterprise capabilities (Eligibility, Coverage Discovery, QA Rules, Policy Retrieval, Notifications, etc.) are exposed through standardized, governed tool interfaces (e.g., MCP-style tool endpoints).
- **Model Plane (Shared):** Centralizes **model registry, routing, policy enforcement, cost controls, and observability**, allowing federated agents and orchestrators to consume the *right model for each task* without owning model hosting, scaling, or governance. Leverages UAIS to host base and custom approved models.
- **Agent Plane (Federated):** Agents are deployed where they best fit latency, data locality, and team ownership (GCP, AWS, or Azure). Domain orchestrators compose specialist agents based on execution plans. Treating agents as callable services supports reuse across domains and avoids duplicating business logic. Agents can run anywhere (GCP/AWS/Azure) and remain reusable through a common agent invocation interface (e.g., A2A calls).

To reconcile governance with multi-cloud realities, we recommend a **two-level orchestration** model:

- A **Global Orchestrator** in Azure/Foundry acts as the *governed control plane* for cross-domain routing, policy enforcement, and standardized response shaping.
- **Domain Orchestrators** run in the cloud closest to their specialist agents and data (e.g., Pre-Service orchestrator in GCP), executing the detailed plan and invoking shared tools.

Architecture Overview

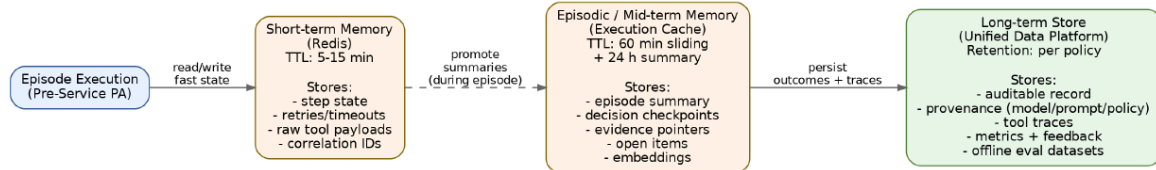


Memory and State Management

Agentic workflows require a layered memory model. Fast step coordination needs short-lived execution state, multi-turn continuity needs an episodic store, and compliance and improvement require durable storage. A layered memory strategy ensures performance and compliance:

- **Short-term memory (Redis):** fast execution scratchpad (minutes). It stores step state, retries/timeouts, raw tool payloads, and correlation IDs.
- **Episodic/mid-term memory (Execution Cache):** within-episode state and evidence continuity (60 minutes → summary up to 24 hours). It stores evolving summaries, decision checkpoints, evidence pointers, open items, and embeddings that support quick retrieval of 'what we decided and why'. It should store summaries and references rather than raw PHI.

- **Long-term store (Unified Data Platform):** durable audit record + analytics + learning (retention per compliance). It retains auditable episode records, provenance (model/prompt/policy versions), tool traces, metrics, and feedback signals. It enables decision replay for audits, operational analytics, offline evaluation, and continuous improvement.

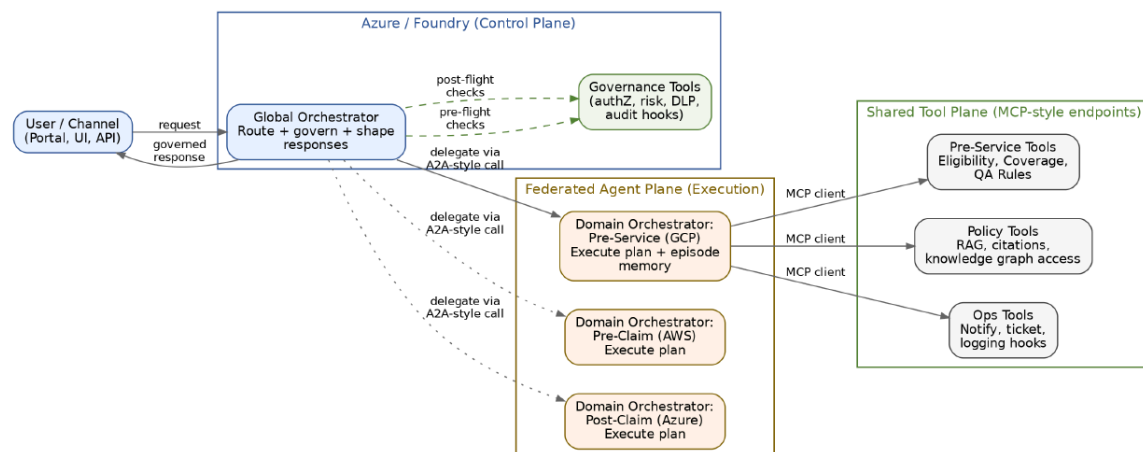


TTL Strategy for Episodic Memory

Episodic Memory will need to be flushed periodically, i.e. every 60 minutes. Before flushing episodic memory the data should be sent to the AI Platform's Long Term Memory store. Sending this data will be done by posting the data to a Queue, the platform will process the data from the Queue and write to the Long-term store.

Multi-Cloud Strategy

When agents are developed and deployed across multiple clouds, a two-level orchestration model balances centralized governance with domain autonomy. A Global Orchestrator provides routing and governance; Domain Orchestrators execute domain plans close to agents and data.



Global Orchestrator (Azure/Foundry)

Responsibilities:

- cross-domain routing (intent → domain)
- pre-flight checks (entitlements, allowed actions/tools, risk tier, budgets)
- delegation envelope creation (minimal context + constraints)
- post-flight checks (redaction, formatting, provenance completeness)
- consistent UX and response standards across domains
- cross-domain observability correlation

Design:

1. Receive request with identity context.
2. Run governance pre-flight (entitlements, PHI classification, budgets).
3. Route to a domain orchestrator endpoint.
4. Provide constraints (allowed tools, required evidence, time/cost budgets).
5. Receive structured result + evidence map.
6. Apply post-flight checks and response shaping.
7. Return consistent response; emit standardized logs/traces.

Domain Orchestrator: “execute the plan”**Responsibilities:**

- Plan execution and step orchestration within the domain.
- Collaboration with specialist agents.
- Tool invocation through the shared Tool Plane.
- Episodic and Short-Term memory management and selective re-execution.
- Domain-specific error handling and remediation logic.

What to measure

- end-to-end episode latency (p50/p95)
- tool latencies + error rates
- retrieval relevance metrics (e.g., top-k hit quality)
- token usage and cost per episode

- cache hit rates (Redis + episodic memory)
- frequency of remediation triggers
- human-review rate and override reasons

Propagate a single `trace_id` across:

- global orchestrator
- domain orchestrator
- specialist agents
- tool calls
- retrieval calls

Model Plane (Shared)

A centralized, governed model service layer that provides:

- **Model Registry** (owner, capability tag, domain tag)
- **Model Router/Gateway** (a consistent API for inference and embeddings)
- **Access to models** through the model gateway with a standardized API
- **Policy & Safety Controls** (PHI rules, region constraints, cost budgets, logging)

Benefits

A) Faster time to production

- Domain agents can start with **approved models** immediately.
- Common auth, telemetry, guardrails, retries, redaction policies.

B) Better reliability and scale

- Central router can implement:
- fallback models when primary is degraded
- timeouts + circuit breakers
- capacity management and rate limiting

C) Cost optimization without domain rewrites

- Many agent steps don't require a frontier LLM:
- classification, extraction, templated summarization → smaller/cheaper models
- complex reasoning → stronger models
- A router can automatically apply cost/latency policies:

D) Consistent governance and audit

- Platform ensures:
- what model was used
- which version
- what policy profile applied
- what was blocked/redacted

E) Safe reuse of custom/fine-tuned models

- Domain teams can build custom models, but instead of each team inventing distribution:
- register the model in the AI Platform **Model Registry**
- pass through shared logging, security, and evaluation gates
- allow other domains to reuse it

Capabilities

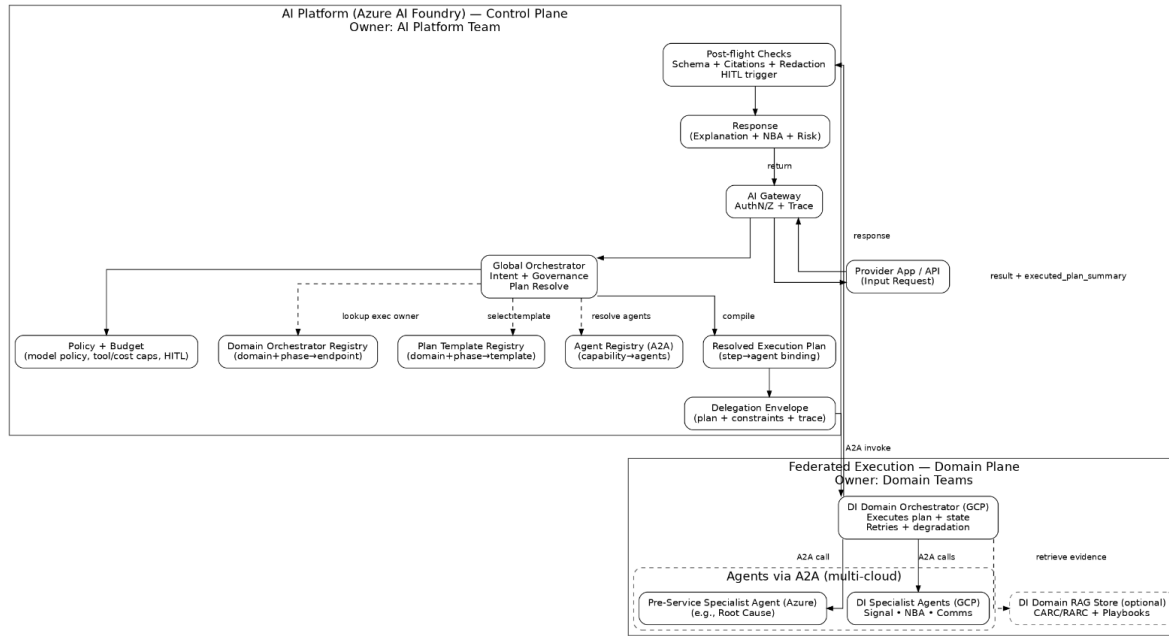
- Standard Interface
- /classify – determine request intent, route to correct domain orchestrator
- /extract – entity and fact normalization
- /embed – grounding and retrieval
- /rerank – evidence precision
- /generate – reasoning, explanation, decision rationale, next steps

API	Use When	Do NOT Use When
/classify	Routing, intent detection, risk/safety gating	Complex reasoning or explanations

API	Use When	Do NOT Use When
/extract	Converting text into structured fields or checklists	Producing narrative explanations
/embed	Semantic search, RAG, similarity matching	Direct decision-making or summarization
/rerank	Selecting best evidence from retrieved candidates	Creating new content or reasoning
/generate	Reasoning, planning, summarization, explanation	Deterministic extraction or simple routing

Runtime Execution Flow

1. App/API request enters the Global Orchestrator through HCP AI Gateway.
2. Global Orchestrator classifies intent and phase.
3. Governance pre-flight executes (budgets, model policy, HITL rules).
4. A Declarative Plan Template is selected from the registry.
5. Steps are resolved to concrete agents via Agent Registry (A2A).
6. A Delegation Envelope is created and sent to the Domain Orchestrator.
7. Domain orchestrators execute the plan locally, invoking agents cross-cloud via A2A.
8. Results return to the Global Orchestrator for post-flight checks.
9. Response is normalized, audited, and returned to the caller.
10. The Global Orchestrator decides what should run and under what constraints.
11. The Domain Orchestrator decides how to actually run it correctly.



What it aims at

- Need to evolve workflows without redeploying domain code.
- Business users should be able to **visualize**, **govern**, and **safely modify** agentic workflows—such as replacing one specialist agent with another—while preserving domain autonomy, compliance, and auditability.

Declarative Plan Templates (DPT)

A **Declarative Plan Template** is a **versioned, machine-readable description of a domain workflow** that specifies:

- the **logical steps** of a workflow,
- the **capabilities required** at each step,
- the **constraints and policies** that must be enforced,
- but **not the concrete agent implementation**.

At runtime:

- the **Global Orchestrator (Azure AI Foundry)** selects and resolves a plan template,
- binds each step to a concrete agent discovered via **A2A** and the **Agent Registry**,

- injects budgets, policies, and HITL rules,
- and delegates execution to the **Domain Orchestrator** (e.g., DI running in GCP).

Components

Storage

Declarative Plan Templates should be stored as **versioned YAML or JSON artifacts** in a Git-backed repository managed by the AI Platform team. This provides:

- full change history and rollback,
- peer review and approval workflows,
- auditability required in regulated environments.

Runtime Registry

A lightweight **Plan Template Registry Service** (deployed in Azure alongside Foundry) serves as the runtime lookup layer for the Global Orchestrator. It exposes APIs such as:

GET /plans/{plan_id}

GET /plans/latest?domain=di&phase=post_claim

Only approved and active versions are returned at runtime.

Foundry UI

- a graph-based visualization of plan templates,
- step-by-step inspection of required capabilities,
- integration with the Agent Registry to show available agents per step,
- controlled editing to create new plan versions (draft → approved → active).

Sample Plan Template (e.g. for DI)

```
1 plan_id: di.post_claim.workflow.v1
2 domain: denial_intelligence
3 phase: post_claim
4 description: Explain denial root cause and recommend next actions
5 steps:
6   - step_id: extract_signals
7     capability: denial_signal_normalization
   - step_id: determine_root_cause
```

```

8     capability: denial_root_cause_analysis
9     - step_id: recommend_nba
10    capability: next_best_action
11    - step_id: explain_for_provider
12    capability: provider_friendly_explanation
13

```

A2A Agent Registry:

All agents register in a shared Agent Registry with metadata as below (can be extended)

```

1  {
2    "agent_id": "pre_service.root_cause.agent.v2",
3    "capabilities": ["denial_root_cause_analysis"],
4    "domain": "pre_service",
5    "a2a_endpoint": "https://ps-agent.azure.com/a2a",
6    "policies_supported": ["citations_required"],
7    "latency_profile_ms": 500
8  }

```

When resolving a plan:

- the Global Orchestrator matches **capability + policy compatibility + budget constraints**,
- selects the most appropriate agent,
- and binds it into the **Resolved Execution Plan**.

Domain Orchestrator Registry:

Purpose:

Domain Orchestrator Registry stores details of which orchestrator owns which execution.

“Given the domain and phase, which orchestrator owns execution?”

It does not describe agents, tools, or workflows.

Sample Template:

```

1  {
2    "orchestrator_id": "di.orchestrator.v1",
3    "domain": "denial_intelligence",
4    "supported_phases": [
5      "inquiry",
6      "final_837",

```

```
7     "post_claim"  
8   ],  
9   "a2a_endpoint": "https://di-orchestrator.gcp.company.com/a2a",  
10  "environment": "azure",  
11  "trust_tier": "PROD_CERTIFIED",  
12  "supported_plan_versions": [  
13    "di.post_claim.workflow.v1",  
14    "di.post_claim.workflow.v2"  
15  ],  
16  "capabilities": [  
17    "denial_explanation",  
18    "denial_risk_prediction",  
19    "denial_recommendation"  
20  ],  
21  "status": "ACTIVE",  
22  "priority": 10,  
23  "metadata": {  
24    "owner_team": "DI Platform Team",  
25    "on_call": "di@optum.com"  
26  }  
27 }
```

HITL Mechanism

Layer 1 — Global baseline policies (platform-owned)

Applies to *all* domains.

Examples:

- Any high-dollar decision requires review
- Any budget exhaustion triggers review
- Any action execution in provider UI requires review
- Any model policy violation triggers review
- These are non-negotiable enterprise controls.

Layer 2 — Domain-specific overlays (domain-authored, platform-owned)

Each domain defines domain overlays that refine the baseline.

Examples:

- DI:
- confidence < 0.6 → conditional HITL
- multiple root causes → HITL
- Pre-Service:
- missing auth + high-cost service → HITL
- Claims:
- predicted denial probability > 0.7 → HITL

Example of a policy structure:

```

1  policy_id: hitl.di.post_claim.v1
2  domain: denial_intelligence
3  phase: post_claim
4  status: ACTIVE
5  version: 1.0
6  baseline_rules:
7    - name: high_value_claim
8      condition: claim_amount > 50000
9      outcome: MANDATORY_REVIEW
10   - name: budget_exhausted
11     condition: budget_exhausted == true
12     outcome: MANDATORY_REVIEW
13 domain_rules:
14   - name: low_confidence
15     condition: confidence < 0.6
16     outcome: CONDITIONAL_REVIEW
17   - name: weak_evidence
18     condition: evidence_strength == "LOW"
19     outcome: CONDITIONAL_REVIEW
20   - name: multiple_root_causes
21     condition: domain_risk_flags contains "MULTIPLE_ROOT_CAUSES"
22     outcome: MANDATORY_REVIEW
23 channel_modifiers:
24   provider_ui:
25     tighten: true
26   internal_ops:
27     relax: true
28 default_outcome: AUTO_APPROVED

```

Observability and Operations

What to measure - end-to-end episode latency (p50/p95) - tool latencies + error rates - retrieval relevance metrics (e.g., top-k hit quality) - token usage and cost per episode - cache hit rates (Redis + episodic memory) - frequency of remediation triggers - human-review rate and override reasons

Propagate a single trace_id across: - global orchestrator - domain orchestrator - specialist agents - tool calls - retrieval calls

Risks and Mitigations

- Cross-cloud latency and cost: Mitigate by executing domain orchestration close to agents/data and limiting global orchestration to routing/governance.
- Inconsistent tool schemas: Mitigate with strict contracts and gateway enforcement; return evidence-friendly reason codes.
- Stale episodic context: Mitigate with TTL decay, invalidation triggers, and policy version pinning per episode.
- Privacy leakage: Mitigate with summary-first storage, redaction, and tokenized references across boundaries.
- Policy drift: Mitigate with offline replay tests and release gates for policy, prompt, or tool changes.

FAQs

1. Once Global Orchestrator finds suitable Domain orchestrator based on the intent, why does it still needs to resolve the plan template and not just redirect the call to the domain orchestrator for execution?

Because “finding the right domain orchestrator” answers only who owns execution. It does not answer what to run, which agents/tools to use, what policies apply, or what limits must be enforced.

So the sequence is intentionally: Route (who) → Resolve (what/how under constraints) → Delegate (execute)

Plan resolution is how the platform enables cross-domain agent reuse

2. If Domain workflow has specialized agents and Domain Orchestrator needs to refer those agents to, how does global orchestrator makes sure that those

agents will be picked up while resolving the plan template. Can workflow be a mix of deterministic (business logic in API registered as MCP tools) and agentic?

Yes—plan resolution still works even when a DI workflow is a mix of deterministic tool calls (MCP) and agentic steps, and you can also ensure the resolver picks the three DI-owned specialized agents (not “random” agents from other domains) by encoding ownership + binding constraints into the template and agent registry metadata.

Mechanisms to guarantee of selection of specialized agents:

Use constraint-based selection, not “best match” alone. You have 3 strong options (can combine):

A) Allowed agent pool / domain allowlist

In the plan template, specify a selection constraint such as: - allowed_domains: ["denial_intelligence"] - or allowed_agent_groups: ["di-certified"]

This forces the resolver to consider only DI-owned agents for those steps.

B) Binding by “agent role”

Define stable logical roles within DI: - di.signal.interpretation - di.root.cause - di.nba.recommendation

Then agents register against those roles. Plan steps refer to roles, not generic capabilities.

C) Trust tier / certification label

Require: - trust_tier: PROD_CERTIFIED - certification: DI_PROD This prevents resolving to experimental/other domain agents. Does plan resolution work for deterministic MCP tool calls + agentic steps? Yes—just treat deterministic tool calls as first-class step types in the template, like: - step_type: tool_call (MCP) - step_type: agent_call (A2A) - step_type: decision (rule/classifier) - step_type: compose (final response)

The Global Orchestrator resolves: - agent steps → binds to A2A agents via Agent Registry - tool steps → binds to tool IDs via Tool Registry / MCP Registry (or a “tool catalog”)

Then the Domain Orchestrator executes the whole plan.